Examen de Prueba — Entorno Cliente, Unidad 2 (DWEC)

EXAMEN DE PRUEBA — Entorno Cliente, Unidad 2 (DWEC)

Curso: 2º DAW — Desarrollo Web en Entorno Cliente **Unidad:** 2 — Programación en el cliente con JavaScript **Temas evaluados:** 2.1 · 2.2 · 2.3 · 2.4 · 2.5 · 2.6 **Alumno:** _____ **Fecha:** _____ **Nota:** _____ / 10

Instrucciones generales

- Lee cada ejercicio con atención antes de responder. - En los ejerciones de "¿Qué sale por consola?", indica la salida exacta o escribe `Error` si el código falla. - En los ejercicios de "código erróneo", reescribe el bloque corregido y explica brevemente el fallo. - No uses consola ni navegador durante el examen salvo que el profesor lo indique.

==========

SECCIÓN A — ¿QUÉ SALE POR CONSOLA?

Predice la salida exacta de cada bloque de código.

Escribe "Error" si el script lanza una excepción.

==========

A.1 — Tema 2.2: Operadores y cortocircuito

```
console.log( || "Fallback");
console.log( ?? "Fallback");
console.log( + "5");
let i = ;
console.log(i++ * ++i);
```

Salida esperada: _____

A.2 — Tema 2.2: typeof y comparaciones

```
console.log(typeof n);
console.log(typeof Symbol());
console.log(typeof null);
console.log(typeof NaN);
console.log( == "5");
console.log( === "5");
console.log( - "5");
const alumno = { id , curso "2 DAW", activo true };
console.log("curso" in alumno);
delete alumno.activo;
console.log(alumno instanceof Object);
```

Salida esperada: _____

A.3 — Tema 2.1: ASI (Automatic Semicolon Insertion)

```
function crearConfiguracion() {
  return
  { modo "oscuro", puerto  };
}

const config = crearConfiguracion();
console.log(config);
```

Salida esperada: _____

A.4 — Tema 2.2: operadores unarios combinados

```
let x = ;
let y = ;
let resultado = x++ * ++y + (x & );
console.log("resultado:", resultado);
console.log("x:", x);
console.log("y:", y);
```

Salida esperada: _____

A.5 — Tema 2.2: || vs ?? con valor cero

```
function fijarDescuento(descuento) {
  let tasaFinalA = descuento || ;
  let tasaFinalB = descuento ?? ;
  console.log(Tasa A ${tasaFinalA} | Tasa B ${tasaFinalB});
}
fijarDescuento();
```

Salida esperada: _____

A.6 — Tema 2.1: case-sensitivity

```
let totalVentas = ;
let TotalVentas = ;
console.log(totalVentas);
console.log(TotalVentas);
```

Salida esperada: _____

A.7 — Tema 2.3: var vs let con setTimeout

```
for (var i = ; i < ; i++) {
  setTimeout(() => console.log("var:", i), );
}
for (let j = ; j < ; j++) {
  setTimeout(() => console.log("let:", j), );
}
```

Salida esperada: _____

A.8 — Tema 2.3: Zona Muerta Temporal (TDZ)

```
let usuario = "Carlos";

function configurarPerfil() {
  console.log(usuario);
  let usuario = "Marta";
}
configurarPerfil();
```

Salida esperada: _____

A.9 — Tema 2.6: bucle silencioso con splice

```
const numeros = [, , , , ];
for (let i = ; i < numeros.length; i++) {
  if (numeros[i] %  === ) {
    numeros.splice(i, );
  }
}
console.log(numeros);
```

Salida esperada: _____

A.10 — Tema 2.6: bucles for...in vs for...of

```
const colores = ["rojo", "azul"];
Array.prototype.nuevoMetodo = "basura";

for (const indice in colores) {
  console.log("in:", indice, typeof indice);
}

for (const valor of colores) {
  console.log("of:", valor);
}
```

Salida esperada: _____

==========

SECCIÓN B — PREGUNTAS TIPO TEST

Marca con una X la respuesta correcta.

==========

B.1 — Tema 2.1: Lenguajes de cliente

1. ¿Cuál de los siguientes es el estándar nativo universal del navegador sin necesidad de plugins?

a) ☐ Adobe Flash b) ☐ Java Applets c) ☐ JavaScript (ECMAScript) d) ☐ VBScript

---

B.2 — Tema 2.1: Transpilación con Babel

2. ¿Qué hace Babel cuando recibe código ES6 moderno?

a) ☐ Lo compila a binario máquina b) ☐ Lo traduce a JavaScript ES5 equivalente para navegadores antiguos c) ☐ Lo ejecuta directamente en el servidor d) ☐ Lo convierte a HTML

B.3 — Tema 2.1: Identificadores válidos

3. ¿Cuál de estos identificadores es VÁLIDO en JavaScript?

a) ☐ `1erUsuario` b) ☐ `precio-final` c) ☐ `class` d) ☐ `$elementoDOM`

B.4 — Tema 2.2: typeof

4. ¿Qué devuelve `typeof null`?

a) ☐ `"null"` b) ☐ `"undefined"` c) ☐ `"object"` d) ☐ `"number"`

B.5 — Tema 2.2: Coerción con + y -

5. ¿Qué imprime `console.log(5 - "5")`?

a) ☐ `"55"` b) ☐ `0` c) ☐ `NaN` d) ☐ `10`

B.6 — Tema 2.2: Igualdad estricta

6. ¿Qué operador exige igualdad de valor Y de tipo sin conversiones ocultas?

a) ☐ `==` b) ☐ `===` c) ☐ `!=` d) ☐ `??`

B.7 — Tema 2.2: BigInt

7. ¿Cuál es el sufijo correcto para declarar un BigInt?

a) ☐ `b` b) ☐ `n` c) ☐ `big` d) ☐ `I`

B.8 — Tema 2.2: operadores in / delete / instanceof

8. ¿Qué hace el operador `delete` sobre una propiedad de un objeto?

a) ☐ Elimina la propiedad del objeto b) ☐ Borra todo el objeto c) ☐ Convierte la propiedad en `undefined` d) ☐ Impide modificar la propiedad

---

B.9 — Tema 2.3: Ámbitos var/let

9. ¿Cuál es la diferencia principal entre `var` y `let` en un bucle con `setTimeout`?

a) ☐ `var` es más rápido b) ☐ `let` crea una variable por cada iteración (ámbito de bloque); `var` comparte una sola para todo el bucle c) ☐ `let` no puede usarse en bucles d) ☐ No hay diferencia

---

B.10 — Tema 2.3: TDZ

10. ¿Qué ocurre si lees una variable `let` antes de su declaración dentro de su mismo ámbito?

a) ☐ Devuelve `undefined` b) ☐ Devuelve `null` c) ☐ Lanza `ReferenceError` (Zona Muerta Temporal) d) ☐ Crea una variable global automáticamente

---

B.11 — Tema 2.3: "use strict"

11. Con `"use strict"`, ¿qué ocurre si asignas un valor a una variable no declarada?

a) ☐ Se crea una propiedad en `window` silenciosamente b) ☐ Lanza `ReferenceError` c) ☐ Se ignora la asignación d) ☐ Devuelve `NaN`

B.12 — Tema 2.4: Truthy / Falsy

12. ¿Cuál de estos valores es **truthy** en JavaScript?

a) ☐ `0` b) ☐ `""` (cadena vacía) c) ☐ `"0"` (cero como texto) d) ☐ `undefined`

B.13 — Tema 2.4: Number.isNaN

13. ¿Qué devuelve `Number.isNaN(NaN)`?

a) ☐ `false` b) ☐ `true` c) ☐ `undefined` d) ☐ `NaN`

B.14 — Tema 2.4: Conversiones

14. ¿Qué devuelve `Number(undefined)`?

a) ☐ `0` b) ☐ `NaN` c) ☐ `1` d) ☐ `undefined`

B.15 — Tema 2.5: Guard clauses

15. ¿Qué es una "guard clause" (comprobación al principio)?

a) ☐ Un `if` al final de la función b) ☐ Una comprobación de errores al inicio que devuelve temprano si algo falla c) ☐ Un bucle infinito d) ☐ Un comentario al principio del archivo

---

B.16 — Tema 2.6: Bucles

16. ¿Qué bucle se ejecuta **al menos una vez**, aunque la condición sea falsa al inicio?

a) ☐ `while` b) ☐ `do...while` c) ☐ `for` d) ☐ `forEach`

B.17 — Tema 2.6: forEach

17. ¿Por qué NO se puede usar `break` dentro de un `forEach`?

a) ☐ `forEach` no es un bucle real b) ☐ `forEach` no admite `break` ni `continue`; hay que usar `for...of` c) ☐ Solo funciona en arrays vacíos d) ☐ `break` solo funciona en `while`

B.18 — Tema 2.6: for...in sobre arrays

18. ¿Qué problema tiene usar `for...in` para recorrer un array?

a) ☐ Los índices llegan como cadenas y puede listar propiedades basura del prototipo b) ☐ No funciona en ningún navegador c) ☐ Solo recorre el primer elemento d) ☐ Siempre devuelve los valores en orden inverso

---

==========

SECCIÓN C — HUECOS EN BLANCO

Rellena el código faltante (___).

==========

C.1 — Tema 2.2: Declaración de variables

```
___ nombreCliente = "Ana García";      // variable que puede reasignarse
___ MAX_REINTENTOS = ;                 // constante que no cambiará
let contador = ;                       // ámbito de bloque
```

Respuesta: - Primera línea: ___ → _____ - Segunda línea: ___ → _____

C.2 — Tema 2.2: Operador de coalescencia nula

```
let apellidos = null;
console.log(apellidos ?? ___);   // debería imprimir "Sin apellidos"
```

Rellena el hueco: ___ → _____

C.3 — Tema 2.3: Ámbito de bloque con let

```
{
  ___ localidad = "Madrid";   // sólo existe dentro de este bloque
}
console.log(localidad);        // ¿qué imprime? _____
```

Respuesta: ___ → _____ **Salida del console.log:** _____

C.4 — Tema 2.4: Conversión booleana

```
console.log(Boolean("0"));   // _____
console.log(Boolean([]));    // _____
console.log(Boolean(""));    // _____
```

Rellena las tres salidas: _____ , _____ , _____

C.5 — Tema 2.5: Estructura if / else if

```
let nota = ;

if (nota >= ) {
  console.log("Sobresaliente");
} ___ (nota >= ) {
  console.log("Notable");
} ___ {
  console.log("Insuficiente");
}
```

Rellena: primer hueco ___ → _____ **Rellena:** segundo hueco ___ → _____

C.6 — Tema 2.6: Bucle for determinista

```
for (___ i = ; i <= ; i___) {
  console.log(i);
}
```

Rellena: `___ i = 1` → _____ **Rellena:** `i ___` → _____

C.7 — Tema 2.6: Bucle while (contador manual)

```
let i = ;
while (___ ) {
  console.log(i);
  i++;
}
```

Rellena la condición: ___ → _____ *(el bucle debe imprimir 0, 1, 2, 3, 4)*

==========

SECCIÓN D — CÓDIGO ERRÓNEO PARA CORREGIR

Lee el código, indica el fallo y escribe la versión corregida.

==========

D.1 — Tema 2.1: Sensibilidad a mayúsculas

```
Function calcularDescuento(PrecioBase) {
  Const porcentaje = .;
  If (PrecioBase > ) {
    Alert("Descuento aplicado");
    Return PrecioBase * porcentaje;
  }
  Return ;
}
```

Fallo: _____

Código corregido:

```
_____
_____
_____
_____
```

D.2 — Tema 2.1: ASI (return en línea separada)

```
function crearUsuario() {
  return
  { nombre "Ana", edad  };
}
console.log(crearUsuario());
```

Fallo: _____

Código corregido:

```
_____
_____
_____
```

D.3 — Tema 2.3: Redeclaración con var y let

```
let edad = ;
let edad = ;   // Error
```

Fallo: _____

Código corregido:

```
_____
```

D.4 — Tema 2.4: Comparación booleana confusa

```
if ("0" == false) {
  console.log("Verdadero");
}
```

Fallo / trampa: _____

¿Qué imprime realmente? _____

D.5 — Tema 2.5: else innecesario después de return (no-else-return)

```
function validarEdad(edad) {
  if (edad < ) {
    return "Edad inválida";
  } else {
    return "Edad válida";
  }
}
```

Fallo: _____

Código corregido (guard clause):

```
function validarEdad(edad) {
_____
_____
}
```

D.6 — Tema 2.6: Bucle silencioso con splice (error de lógica)

```
const numeros = [, , , , ];
for (let i = ; i < numeros.length; i++) {
  if (numeros[i] %  === ) {
    numeros.splice(i, );
  }
}
console.log(numeros);   // imprime [4, 8, 9] — FALLO: se salta elementos
```

Fallo: _____

Código corregido (recorrer de atrás hacia adelante):

```
_____
_____
_____
_____
```

D.7 — Tema 2.6: Bucle infinito

```
let i = ;
while (i < ) {
  console.log(i);
  // falta algo
}
```

Fallo: _____

Código corregido:

```
_____
_____
```

==========

CLAVE DE RESPUESTAS (para el profesor)

==========

SECCIÓN A — ¿QUÉ SALE POR CONSOLA?

A.1: `Fallback`, `0`, `55`, `8` **A.2:** `bigint`, `symbol`, `object`, `number`, `true`, `false`, `0`, `true`, `{ id: 1001, curso: '2 DAW' }`, `true` **A.3:** `SyntaxError: Unexpected token ':'` (el ASI inserta `;` tras `return`, rompiendo el objeto) **A.4:** `resultado: 15`, `x: 4`, `y: 5` **A.5:** `Tasa A: 15 | Tasa B: 0` **A.6:** `500`, `1200` **A.7:** `var: 3`, `var: 3`, `var: 3`, `let: 0`, `let: 1`, `let: 2` **A.8:** `ReferenceError: Cannot access 'usuario' before initialization` **A.9:** `[4, 8, 9]` (al borrar con splice, el array se desplaza y `i++` se salta elementos) **A.10:** `in: 0 string`, `in: 1 string`, `in: nuevoMetodo string`, `of: rojo`, `of: azul`

SECCIÓN B — TIPO TEST

1. c · 2. b · 3. d · 4. c · 5. b · 6. b · 7. b · 8. a · 9. b · 10. c · 11. b · 12. c · 13. b · 14. b · 15. b · 16. b · 17. b · 18. a

SECCIÓN C — HUECOS

C.1: `let`, `const` C.2: `"Sin apellidos"` C.3: `let`, `ReferenceError` (no existe fuera del bloque) C.4: `true`, `true`, `false` C.5: `else if`, `else` C.6: `let`, `i++` C.7: `i < 5`

SECCIÓN D — CÓDIGO ERRÓNEO

D.1. Fallo: palabras reservadas y funciones nativas con mayúscula (`Function`, `Const`, `If`, `Alert`, `Return`). Corregido: todo en minúsculas (`function`, `const`, `if`, `alert`, `return`).

D.2. Fallo: ASI inserta `;` tras `return`, devolviendo `undefined` y dejando el objeto como bloque roto. Corregido: llave `{` en la misma línea que `return`.

D.3. Fallo: `let` no permite redeclarar la misma variable en el mismo ámbito. Corregido: usar `let edad = 20; edad = 25;` o cambiar el nombre.

D.4. Trampa: `"0" == false` es `true` por coerción (`0 == 0`), y `if ("0")` es truthy porque la cadena no está vacía. Imprime: `Verdadero`.

D.5. Fallo: `else` innecesario tras `return`. Corregido:

```
function validarEdad(edad) {
  if (edad < ) return "Edad inválida";
  return "Edad válida";
}
```

D.6. Fallo: al borrar con `splice` en bucle creciente, el array se desplaza y `i++` se salta el siguiente elemento. Corregido:

```
const numeros = [, , , , ];
for (let i = numeros.length - ; i >= ; i--) {
  if (numeros[i] %  === ) {
    numeros.splice(i, );
  }
}
console.log(numeros);   // [9]
```

D.7. Fallo: falta la actualización `i++`, lo que causa bucle infinito. Corregido:

```
let i = ;
while (i < ) {
  console.log(i);
  i++;
}
```

--- **Fin del examen.** Entrega este impreso al profesor junto con tu hoja de respuestas.